中山大學
SUN YAT-SEN UNIVERSITY



文件操作

中山大学计算机学院

讲课人：潘茂林

目录

CONTENTS

01 文件概述

02 文件基础操作

03 文本文件处理

04 二进制文件处理

05 文件随机读写

文件概述

文件(File)：（狭义地）是指存储在计算机外部介质（如磁盘等）上的文字序列或二进制字节的序列。

磁盘文件指保存在磁盘上的文件，它有唯一的**全称**，例如："/sources/hello.c"，其中"/sources/"是文件**路径**/*path*，"hello.c" 是**文件名**/*name*；文件**长度**用字节数表示。

文件的用途：

1. **持久保存**。例如编写的源代码，编译产生的中间件结果和执行文件能长期保存；
2. **数据共享**。多个程序运行时可以使用相同的数据，或某程序产生数据供其他程序使用等；
3. **数据与程序分离**。数据的改变不会导致程序重新编译、运行。

文件概述

以数据为中心视角，文件的分两大类：

文本文件(text)：由ASCII码及其扩展集构成字符构成的文字序列。

- 可以使用任何文本编辑软件打开，如记事本，Sublime，vs code 等等；
- 编码属性，如 ascii，ansi（含 GBK，BIG5等），utf-8等；
- 若其中包含 EOT(4), STX(3), -1 等控制符在部分系统中会导致文件传输中止。

二进制文件：以字符数组格式存放的数据。

- 每类应用有独特的数据，例如 .zip只能由压缩解压程序处理，.obj只能由编译程序使用；
- 通常是将内存中数据格式原样输出到文件中，例如整数会以4个字节补码格式存储；
- 保存 struct 结构体时需要考虑 packed 或 unpack 情况，在空间和性能之间平衡。

文件概述

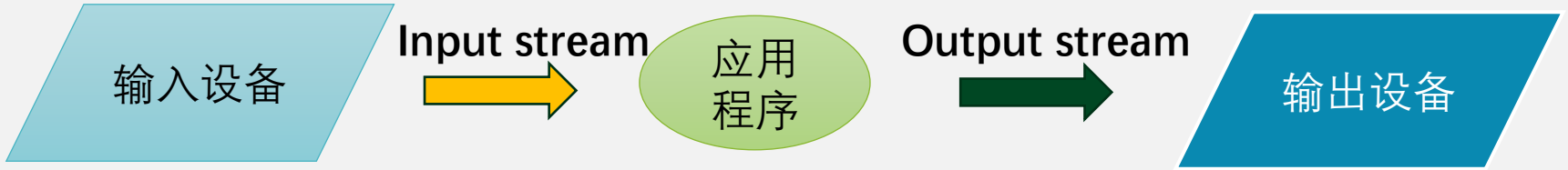
以应用为中心的视角，文件分类：

输入流(stream)：字符或数据通过类似管道的机制流入应用的字节流。流出应用的称为**输出流**。

能产生输入流的设备称为输入设备，如键盘、鼠标、传感器等等。

同样，显示器、打印机、控制器等称为输出设备。

如磁盘等既能产生输入，又接受输出，则称为输入输出设备。

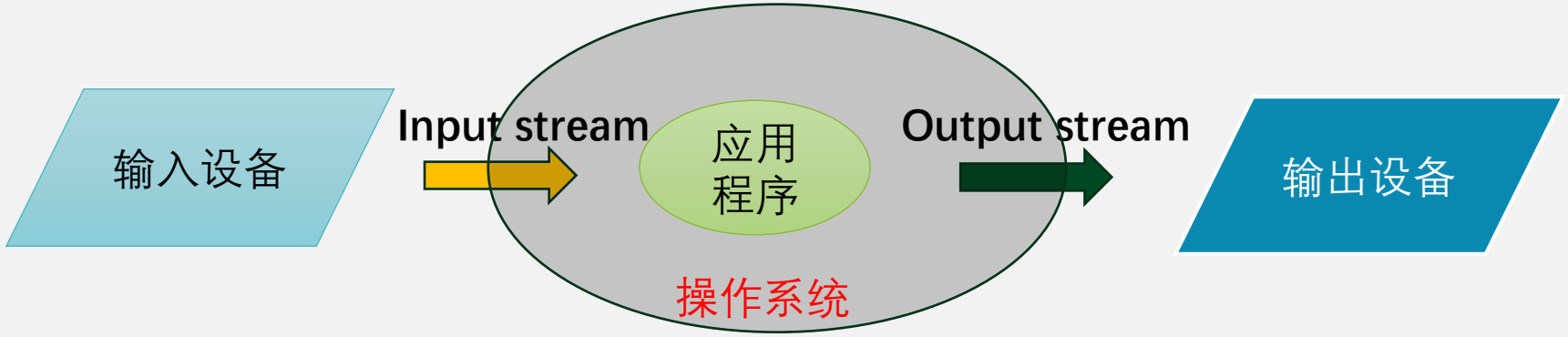


文件概述

以操作为中心的视角，文件操作是操作系统提供的关于流的**通用操作**:

应用程序不能直接操作输入或输出设备。必须通过**操作系统**完成文件操作。

1. **打开文件**：通过一个文件标识符（文件名）建立设备与应用之间的流机制。
2. 操作文件：**读文件**，从流管道中**顺序读入**文字或字节；**写文件**，将字符串或字符数组**顺序输出**到流管道中。
3. **关闭文件**：通知操作系统关闭流机制，释放该文件占用的各种资源。



文件基础操作（1）

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

1. **结构体 FILE**：(不是 struct FILE)描述了控制与管理 IO 流需要的信息。应用程序不建议直接操作结构体，应使用 FILE * 指针通过相关函数操作文件。
2. **预定义标准流**：
   - **stdin** 是 FILE * 变量，标准输入流。默认是控制台键盘输入
   - **stdout** 是 FILE * 变量，标准输出流。默认是控制台输出显示设备
   - **stderr** 是 FILE * 变量，标准错误输出流。默认是控制台输出显示设备
   - 注意：预定义标准流是操作系统为应用程序准备好的流，应用程序一般不需要关闭或打开，可直接读入或输出。例如：scanf 使用 stdin 读入数据，printf 使用 stdout 输出文本。

文件基础操作（2）

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

3. 函数（红色需要掌握，黑色了解）：

- 文件访问：fopen，fclose，freopen，fflush，…
- 直接输入/输出：fread，fwrite，…
- 无格式文本输入/输出：fgets，fputs，fgetc/getc，fputc/putc，ungetc；标准输入/输出：getchar，putchar，gets，puts；
- 格式化文本输入/输出：fscanf，fprintf；标准输入/输出：scanf，printf；字符串输入/输出：sscanf，sprintf；…
- 文件位置：fseek，ftell，fgetpos，fsetpos，rewind；

文件基础操作（3）

头文件<stdio.h>给出了C语言与操作系统交互的函数与数据结构：

3. 函数（红色需要掌握，黑色了解）：

- 错误处理：clearerr，feof，ferror，perror；
- 文件操作：remove，rename，tmpfile，tmpnam；

4. 宏常量

- EOF

文件基础操作 - 步骤

C语言中文件操作一般有四个步骤:

1. **定义处理文件的变量**:
   - 例如: FILE *dest, *src;
2. **打开文件**:
   - 例如: dest = fopen("1.txt", "w");
   - **注意: 必须检查文件指针是否为空**
3. **读写文件**, 或调整读写位置。**注意文本与二进制读写的区别。**
4. **关闭文件**:
   - 例如: fclose(dest);

文件基础操作 - 案例研究

复制磁盘上的文件:

任何磁盘文件可看作字符数组

以写方式与dest文件建立流d

如果有错误，文件指针为 NULL

以读方式与scr文件建立流s

当文件没读完

从流s顺序读最多4K字节到buf

将buf中cnt个字节顺序写入流d

关闭流s和d

```
/*file copy*/
int FileCopy(char *dest, char *scr) {
    char buf[1024*4];

    FILE *d = fopen(dest,"wb");
    if (!d) return -1;
    FILE *s = fopen(scr,"rb");
    if (!s) return -1;

    while (!feof(s)) {
        size_t cnt = fread(buf,1,sizeof(buf),s);
        fwrite(buf,1,cnt,d);
    }

    fclose(s);
    fclose(d);
    return 0;
}
```

文件基础操作 - 打开与关闭文件

打开文件:

```
FILE *fopen( const char *filename, const char *mode );
```

- 打开 filename 所指示的文件，并返回指向关联到该文件的文件流的指针。 mode 用于确定文件访问模式。
- 若错误返回空指针，可通过外部变量 errno 获得错误编号，strerror() 获得错误文本。

关闭文件:

```
int fclose( FILE *stream );
```

- 关闭给定的文件流。冲入任何未写入的缓冲数据到 OS 。舍弃任何未读取的缓冲数据。

文件基础操作 - 打开方式

打开文件方式： mode 字符串

文件访问 模式字符串	含义	解释	若文件已存在的动作	若文件不存在的动作
"r"	读	打开文件以读取	从头读	打开失败
"w"	写	创建文件以写入	销毁内容	创建新文件
"a"	后附	后附到文件	写到结尾	创建新文件
"r+"	读扩展	打开文件以读/写	从头读	错误
"w+"	写扩展	创建文件以读/写	销毁内容	创建新文件
"a+"	后附扩展	打开文件以读/写	写到结尾	创建新文件
文件访问模式标签 "b" 可以可选地指定以二进制模式打开文件。此标签仅在 Windows 系统上生效。 在附加文件访问模式下，数据被写入到文件尾，而不考虑文件位置指示器的当前位置。				
当文件以更新模式打开时（'+' 作为上述模式参数值列表的第二或第三个字符），访问流可以进行输入和输出。然				

文件基础操作 - 直接读或写

读数据：

```
size_t fread( void *buffer, size_t size, size_t count, FILE *stream );
```

- 从给定输入流 stream 读取至多 count 个 size 字节的对象到数组 buffer 中。
- 成功读取的对象数，若出现错误或文件尾条件，则可能小于 count 。
- fread 不区别文件尾和错误，而调用者必须用 feof 和 ferror 鉴别出现者为何。

写数据：

```
size_t fwrite( const void *buffer, size_t size, size_t count, FILE *stream );
```

- 写 count 个来自给定数组 buffer 的对象到输出流stream。
- 成功写入的对象数，若错误发生则可能小于 count 。

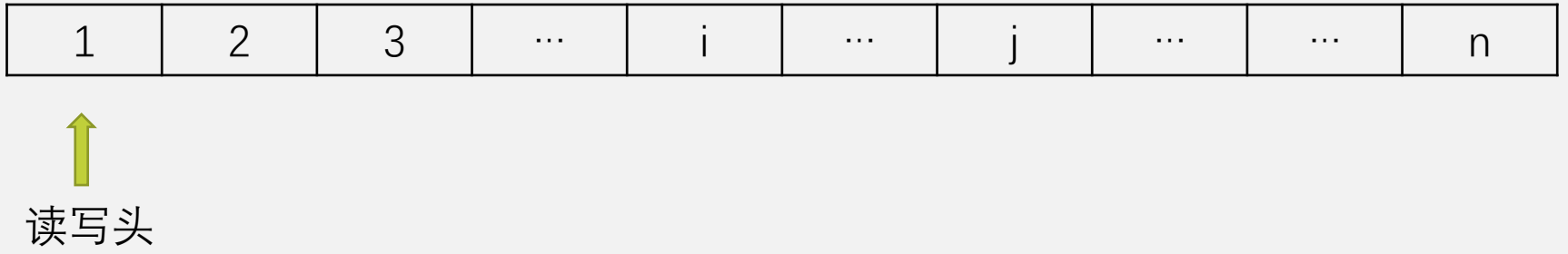
文件基础操作 - 文件结束？

检查文件结束:

```
int feof( FILE *stream );
```

检查是否已抵达给定文件流的结尾。

知识：对于**磁盘文件**，可以看作二进制的动态字符数组。当文件以读方式打开时，有一个读写装置放在待读字节的位置（开始位置），当收到读指令，则将当前字节读了流，然后下移一格顺序读，当读完后指向添加位置时，则就是所谓文件尾部，这时 feof 返回为非零值。



文本文件操作

文本文件具有通用、易于阅读、编辑方便等优势，在 Unix 操作系统中，用户名、网络配置等都采用文本文件保存。文本流处理在C语言标准库中比较丰富。

1. **文本由行构成，因此文本流默认是行缓冲，即遇到换行符文本处理函数才会启动**：
   - 例如：getchar 虽是逐一读入字符，但字符流保存在行缓冲中直到换行或流结束；
2. **在绝大数情形下，文本流仅需要顺序读写，即打开模式是“r”，“w”，“a”之一**：
   - 例如：dest = fopen("1.txt", "a"); 在1.txt文件后添加内容；
3. **部分文本流可能不存在结束，即 feof() 函数永远为假：**
   - 例如：键盘输入流；
4. 文本文件操作分为**有格式文本操作函数**和**无格式文本操作函数**。

文本文件操作 - 有格式文本

格式化文本由一行或多行描述一个数据条目，如学生信息。每个数据条目中包含若干数据域，数据域之间用空白字符分隔。例如：

```
/*text formated*/
int main() {
    FILE *pf = fopen("student.txt","r");
    if (!pf) {
        perror("Open File:");
        exit(9);
    }
    //练习1：这里有三个版本的 stReader，它们都对吗？
    //练习2：将参数改为 stdin 会发生什么？
    stReader(pf);
    fclose(pf);
}
```

```
void stReader(FILE *pf) {
    printf("学生信息：\n");
    int id;
    char name[32],college[32];
    double score;
    while (!feof(pf)) {
        fscanf(pf,"%d%s%s%lf",
            &id,name,college,&score);
        printf("学号:%d 姓名:%s 学院:%s 成
绩:%6.2lf\n",
            id,name,college,score);
    }
}
```

文本文件操作 - 格式化读或写

读数据：

```
int fscanf( FILE *stream, const char  *format, ... );
```

- 熟悉的 scanf 等价于 **`fscanf( stdin, format, ... );`**。
- **返回值**：成功接收参数的数量（可以为零），或者若输入首个参数赋值前发生失败则为**EOF** 。
- 如何从键盘这类无尽文本流读数据，必须思考合适的终止方法。

写数据：

```
int fprintf( FILE *stream, const char *format, ... );
```

- 熟悉的 printf 等价于 **`fprintf( stdout, format, ... );`** 。
- **返回值**：输出的字符数，或若出现输出错误或编码错误则为**负值**。

文本文件操作 - 无格式文本

无格式化文本是看作由**字符串**或**字符**构成的文本。这类文本通常先读入内存，再进行处理。例如：

```
/*text normal*/
int main(void) {
    FILE* tmpf = tmpfile();
    fputs("Alan Turing\n", tmpf);
    fputs("John von Neumann\n", tmpf);
    fputs("Alonzo Church\n", tmpf);

    rewind(tmpf);

    char buf[8];
    while (fgets(buf, sizeof buf, tmpf) != NULL)
          printf("\"%s\"\n", buf);

    if (feof(tmpf))
       puts("End of file reached");
}
```

文本文件操作 - 字符串读或写

读数据：

```
char *fgets( char *str, int count, FILE *stream );
```

- 从文件流读取最多 count - 1 个字符存储于 str 字符数组。若遇文件尾或换行符则终止分析。
- **返回值**：成功时 str，失败时 NULL 。
- 注意：**gets(str)** 与 **fgets(str, count, stdin);** 的语义区别。

写数据：

```
int fputs( const char *str, FILE *stream );
```

- 熟悉的 **puts** 等价于 **fputs(str, stdout);** 。
- **返回值**：成功时，返回非负值。失败时，返回 EOF 并设置错误指示。

文本文件操作 - 字符读或写

读数据：

```
int fgetc( FILE *stream );
```

- 熟悉的 getchar （非getc）等价于 **fgetc(stdin);**。
- 从给定的输入流读取下一个字符。
- **返回值**：成功时为作为 unsigned char 获得并转换为 int 的字符，失败时为 EOF 。

写数据：

```
int fputc( int ch, FILE *stream );
```

- 熟悉的 putchar 等价于 **fputc(stdout);** 。
- **返回值**：成功时返回写入的字符。失败时返回 EOF 。

文本文件操作 - 案例 - 字符读写与错误处理

```
/*text error*/
int main(void) {
    FILE* fp = fopen("test.txt", "r"); //请自己创建测试文本文件，写一些汉字试试!
    if(!fp) {
        perror("File opening failed");
        return EXIT_FAILURE;
    }

    int c; // 注意：int，非char，要求处理EOF
    while ((c = fgetc(fp)) != EOF) { // 标准C I/O读取文件循环
       putchar(c);
    }

    if (ferror(fp))
        puts("I/O error when reading");
    else if (feof(fp))
        puts("End of file reached successfully");

    fclose(fp);
}
```

二进制文件操作

二进制文件通常指程序把内存中数据直接按特定顺序结构输出到磁盘。二进制文件基本操作只有 fread 和 fwrite 两个操作。

1. **二进制文本是 byte 流，默认按块从磁盘读入或输出，即块缓冲**:
   - 例如：为了确保数据写入磁盘，有时需显式调用 **fflush** 同步流与磁盘文件的内容;
2. **二进制磁盘文件通常数据比较多，不只需要顺序读写，更多时候需要读写其中部分内容，即需要更新模式的支持**:
   - 例如：dest = fopen("1.bin", "ab+"); 在1.bin文件后添加数据，“+”表示可以修改更正前面的数据;

二进制文件操作 - 顺序写数据

```
/*text to bin*/
int stBinWriter(int cnt, Student *st) {
    FILE *pf = fopen("student.bin","wb");
    if (!pf) {
        perror("Open File:");
        exit(10);
    }

    fwrite(&cnt, sizeof(int), 1, pf);
    int result = fwrite(st, sizeof(*st), cnt, pf);
    fclose(pf);
    return result;
}
```

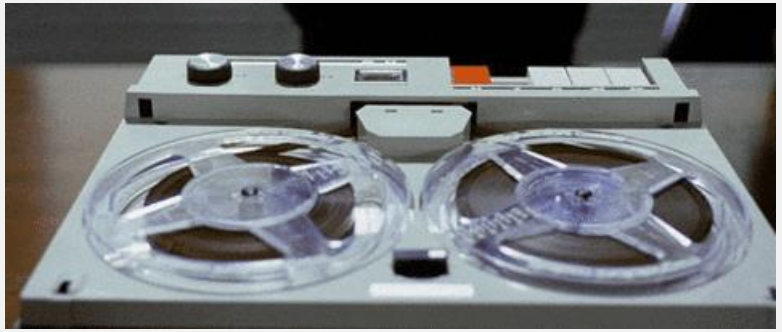
练习:

- 请查看 strudent.txt 和 student.bin 的大小，简述存成二进制文件的优点与劣势。
- 请编写程序读出二进制文件中学生信息。

顺序读写 vs 随机读写

文件操作与磁带操作非常类似。默认的工作模式都是**顺序读写模式**。例如：播放音乐（读）通常会从开始一直播放至结束；录音（写）也是从开始或最后持续的录音。从机电设备运作的角度，顺序读写有着最高的工作效率，通常用 MB/s 为单位衡量设备持续工作性能。

如果要选歌听，或在指定位置录音，则需要先将读写的磁头移到指定位置，再进行部分数据的读/写。这种方式即**随机读写模式**。通常用4KiB/s 为单位度量设备寻找位置并读/写一块（如4KB）数据的性能。



随机读写 - 修改文件位置

寻找文件位置：

```
int fseek( FILE *stream, long offset, int origin );
```

- 功能：设置文件流 stream 的**文件位置指示器**为 offset 所指向的值。
- origin：它能拥有下列值之一： **SEEK_SET** 、 **SEEK_CUR** 、 **SEEK_END**。
- offset：表示相对于文件头（SET）、或尾（END）、或当前位置（CUR）的偏移量。
- 若 stream 以**二进制模式**打开，offset 可用于上述三个位置的偏移量。
- 若 stream 以**文本模式**打开，则 offset 只能为零（任意 origin），或 ftell 值相对 SEEK_SET 。
- 除了更改文件位置指示器， fseek 还撤销 ungetc 的效果并清除文件尾状态，若可应用。
- 返回值：成功时为 0 ，否则为非零。

随机读写 - 修改文件位置

移动到文件首：

```
void rewind( FILE *stream );
```

- 功能：移动文件位置指示器到给定文件流的起始。
- 函数功能等价于 fseek(stream, 0, SEEK_SET);。
- 该函数对二进制方式和文本方式都有效。

随机读写 - 案例研究

选择数据读取：

```
/*random*/
int main() {
    int a[10]={1,2,3,4,5,6,7,8,9,10};

    FILE *fp = fopen( "1.dat", "wb" );
    fwrite(a, sizeof(int), 10, fp);
    fclose(fp);

    fp = fopen( "1.dat", "rb" );
    int b[10] = {0};
    for (int i = 0; i < 5; i++) {
        fread(&b[i], sizeof(int), 1, fp);   //读写完成后，自动移到下一个位置
        fseek(fp, sizeof(int), SEEK_CUR);
    }
    //练习，请在这里添加设当读代码，使得
    // b == {1，3，5，7，9，2，4，6，8，10}
    fclose( fp );

    for (int i=0; i < 10; i++) printf( "%d ", b[i] );
    printf("\n");
    return 0;
}
```

运行程序屏幕将显示：

1 3 5 7 9 0 0 0 0 0

随机读写 - fseek 说明

fseek 函数将文件位置指针定位在某个特定的位置上，从而实现对文件特定位置数据的读/写。

这里文件里面的数值：1 2 3 4 5 6 7 8 9 10，假设只希望读取1、3、5、7、9。

在读取了1后，位置指针自动移动到2的位置上，这时就应该调用fseek改变位置指针使其移动到3的位置上，便可以略过2

随机读写 - 更新模式（+）

更新模式：

- 以更新模式（+）打开文件时，流既可以进行**读操作**又可以进行**写操作**，但同一时刻只能处于读或写的一种状态。
- 如果以 a 或 ab 模式打开文件，则会阻止 fseek 等函数对文件位置指示器的修改。可使用 a+ 或 ab+ 让 fseek 等函数生效。
- **练习**:

1. 修改 random 随机读写程序，使得仅打开、关闭一次，实现相同的功能。
2. 修改 text normal 程序，在文件最后添加一个新人名，并打印所有人名称。注：临时文件默认模式是 **wb+**

中山大學
SUN YAT-SEN UNIVERSITY



谢 谢

中山大学计算机学院

编制人：课题组